Computer Algorithms (Book)

Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekaran

CS-402

ADA

NOTES

UNIT-I

INTRODUCTION TO ALGORITHMS

By: Mr. Dayanand Yadav
Assistant Professor, CSE Dept.

What is an algorithm?

It is a step by step procedure for solving a computational problem.

What is a program?

It is also a step by step procedure for solving a computational problem.

Algorithms vs Program

Algorithm	Program
1. Design	1. Implementation
2. Domain Expert	2. programmer
3. Any Language	3. prog. Lang
4. H/W & OS Independent	4. H/W & OS Dependent
5. Analyze	5. Testing

Characteristics of Algorithm

1. Input — 0 or more
2. Output — atleast one o/p
3. Definiteness — $\sqrt{-1}$ X
4. Finiteness — ∞ X
5. Effectiveness

How to write an Algorithm

Algorithm swap (a, b)

Begin

```
temp = a;
a = b;
b = temp;
```

end

How to analyze an Algorithm

priori Analysis (Before implementation)	posteriori Testing (After implementation)
1. Algorithm	1. Program
2. Language independent	2. Language dependent
3. H/w independent	3. H/w dependent
4. Time & Space function	4. Watch time & Bytes

(4)

How to Analyze an Algorithm

1. Time
2. Space
3. N/W consumption & utilization
4. Power consumption
5. CPU Registers (CPU utilization)

Algorithm swap (a, b)

{

	Time	space
temp = a;	1 unit	a - 1
a = b;	1 unit	b - 1
b = temp;	1 unit	temp - 1
	$f(n) = 3$ units	$S(n) = 3$ words
	$O(1)$	$O(1)$

}

$x = 5 * a + 6 * b$ — 1 (Total 4 statements)

(5)

① Frequency Count Method ①

A =	8	3	9	7	2
	0	1	2	3	4

int n = 5

i = 0
i = 1
i = 2
i = 3
i = 4
i = 5

~~Algorithm Sum (A, n)~~

int main ()
{ int A[] = {1, 2, 3, 4, 5};
int n = 5 ——— 1
int S = 0;
for (i = 0; i ≤ n; i++) — n+1
{ 1 n+1 n
S = S + A [i]; — n
}
printf ("sum = %d", s);
return 0; ——— 1

}

f(n) = 2n+3

O(n)

space

A - n
n - 1
s - 1
i - 1

S(n) n+3

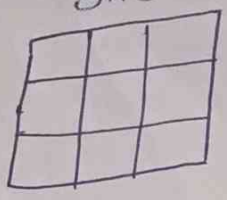
O(n)

(5.1)

(2) Frequency Count Method

$n \times n$

3×3



```
Algorithm Add (A, B, n)
{
 for (i=0; i≤n; i++) ——— n+1
 {
   for(j=0; j≤n; j++) ——— n(n+1)
   {
     c[i][j] = A[i][j] + B[i][j]; ——— n×n
   }
 }
}
```

$f(n) = 2n^2 + 2n + 1$

$O(n^2)$

Space

A — n^2

B — n^2

C — n^2

n — 1

i — 1

j — 1

$S(n) = 3n^2 + 3$

$O(n^2)$

(5.2)

③ Frequency count method

```
p=0;
for(i=1; p<=n; i++)
{
  p = p+i;
}
```

i	p
1	0+1 = 1
2	1+2 = 3
3	1+2+3 = 6
4	1+2+3+4 = 10
.	.
.	.
.	.
K	1+2+3+4+--+K

Assume $p > n$

$\therefore p = \frac{K(K+1)}{2}$

$\frac{K(K+1)}{2} > n$

$K^2 > n$

$\boxed{K > \sqrt{n}}$

$\boxed{O(\sqrt{n})}$

(5.3)

④ Frequency Count Method

```
for (i=1; i<n; i=i*2)
  {
  stmt;
  }
```

i
1
1×2 = 2
2×2 = 2^2
2^2*2 = 2^3
2^3×2 = 2^4
!
)
2^K

Assume $i >= n$

$\because i = 2^K$

$\therefore 2^K >= n$

$2^K = n$

$K = \log_2 n$

$O(\log_2 n)$

(5.4)

(5) Frequency count method

```
for (i=0; i<n; i++)  ——— n
{
  for (j=1; j<n; j=j*2) ——— n*logn
  {
    stmt;  ——— n * logn
  }
}
```

$2n\log n + n$

$O(n \log n)$

Summary

for

(5.5)

Summery

for (i = 0; i < n; i++) —— O(n)

for (i = 0; i < n; i = i + 2) —— O(n)

for (i = n; i > 1; i--) —— O(n)

for (i = 1; i < n; i = i * 2) — $O(\log_2 n)$

for (i = 1; i < n; i = i * 3) — $O(\log_3 n)$

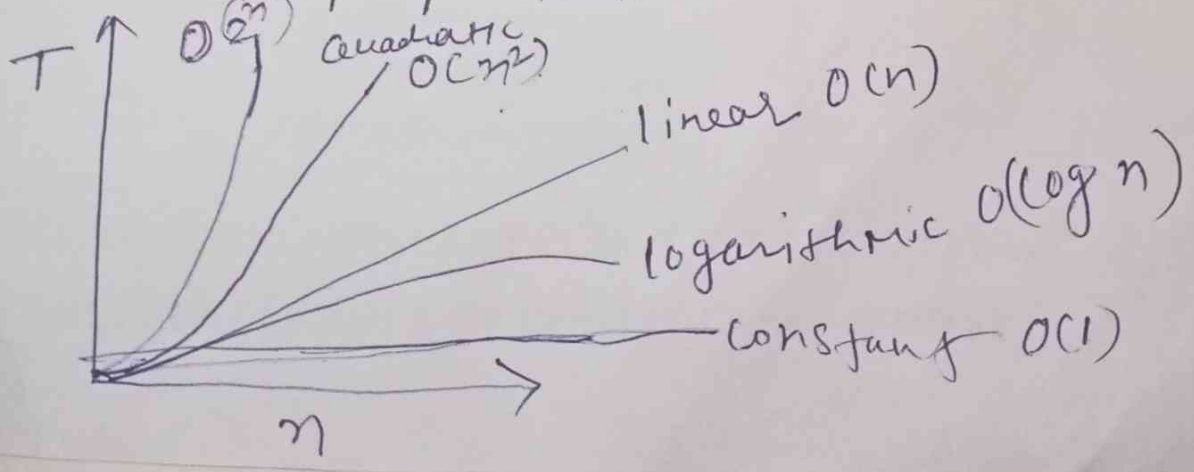
for (i = n; i > 1; i = i/2) — $O(\log_2 n)$

$1 < \log n < \sqrt{n} < n \log n < n^2 < n^3 < \cdots < 2^n < 3^n \cdots < n^n$

Asymptotic Notations

This topic comes from mathematics. To calculate the time complexity of an Algorithm we need a function Thats why we are using the Asymptotic notation to represent time complexity of our algorithm in the form of mathematical notations that is Asymptotic notation.

Asymptotic notations are mathematical notations for describing the behaviour of algorithm and determine the rate of growth of function.



$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \cdots < 2^n < 3^n < \cdots < n^n$

Asymptotic Notations

6.1

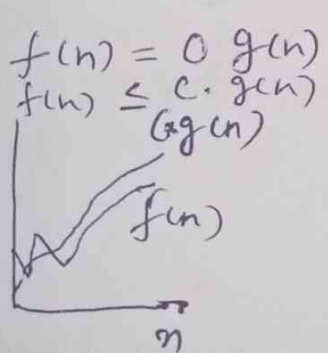
O big-oh (Upper bound)

Ω omega (Lower bound)

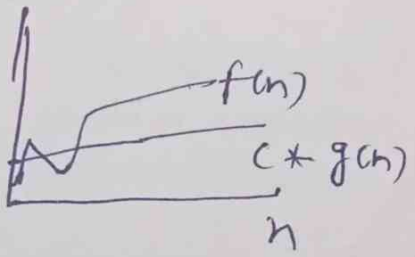
~~(Useful)~~ Θ Theta (Average bound) (~~Tight Bound~~)

---

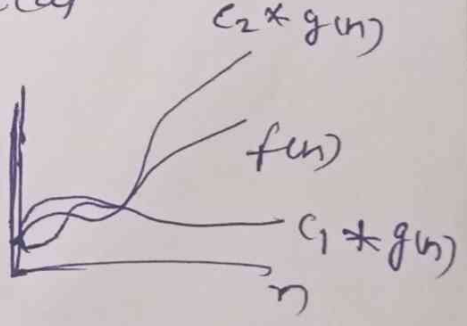
* We write $O(1)$ to mean a computing time that is a constant.
* $O(\log n)$ Logarithmic.
* $O(n)$ is called Linear
* $O(n^2)$ is called quadratic
* $O(n^3)$ is called Cubic
* $O(2^n)$ is called exponential



O
(Upper)



Ω
(Lower)



Θ
(Average)

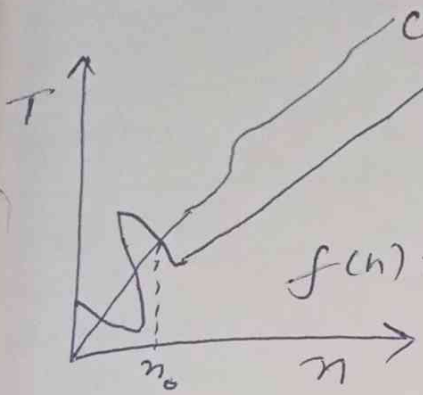
$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \cdots < 2^n < 3^n < \cdots < n^n$

Lower bound Avg bound Upper bound

Big - oh O

6.2

The function $f(n) = O(g(n))$



$f(n) = O(g(n))$

if $\exists$ (there exist) +ve constants c & n_0

such that $f(n) \leq c * g(n)$ $\forall$ $n \geq n_0$

($\forall$ ↑ for all)

e.g.: $f(n) = 2n+3$

$2n+3 \leq 10n \quad n \geq 1$

↑ $f(n)$; ↑↑ $c\ g(n)$

$2n+3 \leq 2n+3n$

$2n+3 \leq 5n$

(OR)

$2n+3 \leq 5n^2 \quad n \geq 1$

$f(n)$ $c\ g(n)$

Closest function

$\therefore f(n) = O(n)$

$\checkmark f(n) = O(n^2)$

$\checkmark f(n) = O(n^3)$

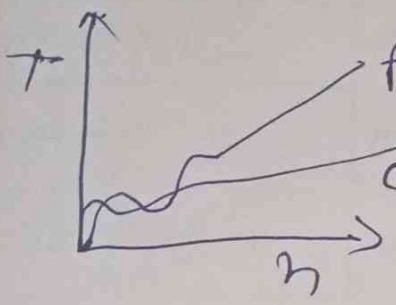
$\checkmark f(n) = O(2^n)$

$\times f(n) = O(\log n)$

Omega Ω (Lower bound)

6.3

The function $f(n) = \Omega(g(n))$



if $\exists$ +ve constants c and n_0

Such that $f(n) \geq c * g(n) \; \forall \; n \geq n_0$

e.g. $f(n) = 2n+3$

$2n+3 \geq 1 * n \; \forall \; n \geq 1$

$f(n)$ ↑ c ↑ $g(n)$

$\therefore f(n) = \Omega(n)$ ✓ closest

$2n+3 \geq 1 * \log n \; \forall \; n \geq 1$

$\therefore f(n) = \Omega(\log n)$

✗ $f(n) = \Omega(n^2)$

Theta notation (Θ) (6.4)

The function $f(n) = \Theta(g(n))$

iff $\exists$ +ve constants $c_1, c_2 \& n_0$

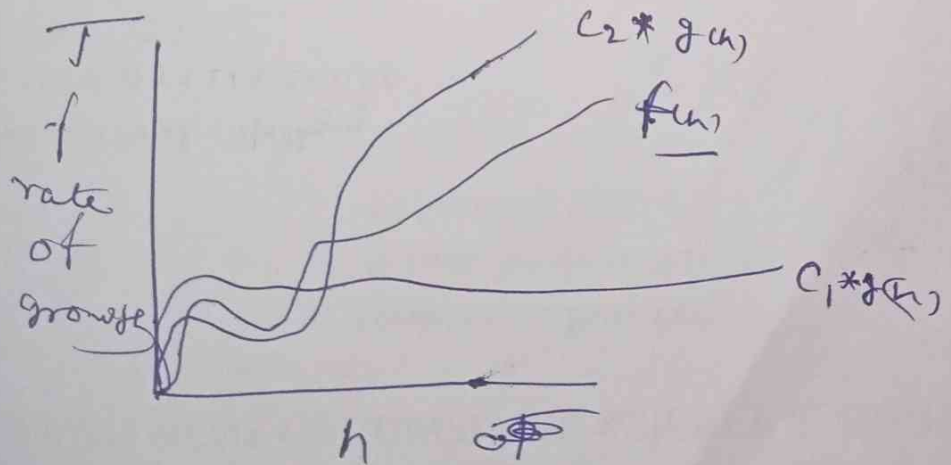
Such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$

e.g. $f(n) = 2n+3 \quad \therefore f(n) = \Theta(n)$

$1 \times n \leq 2n+3 \leq 5 \times n$

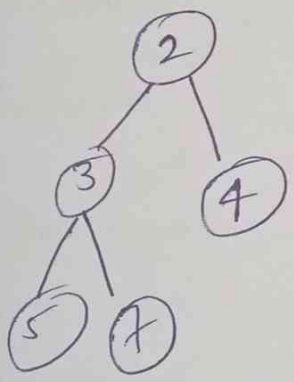
$c_1\, g(n) \quad f(n) \quad c_2\, g(n)$

Most recommended notation

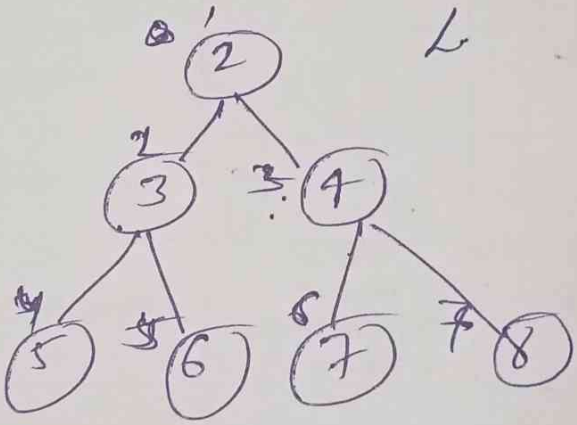


Heap

A Heap is a Complete binary tree data structure that satisfies the heap property: in a min-heap the value of each child is greater than or equal to its parent and in max-heap, the value of each child is less than or equal to its parent.



Min Heap



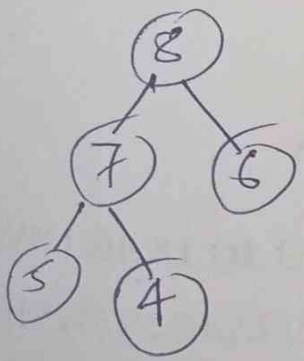
Min heap

left child = 2×i

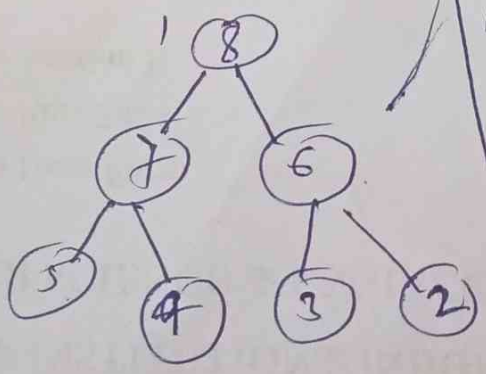
Right child = 2×i+1

2×i+1

Parent = $\left\lfloor i/2 \right\rfloor$



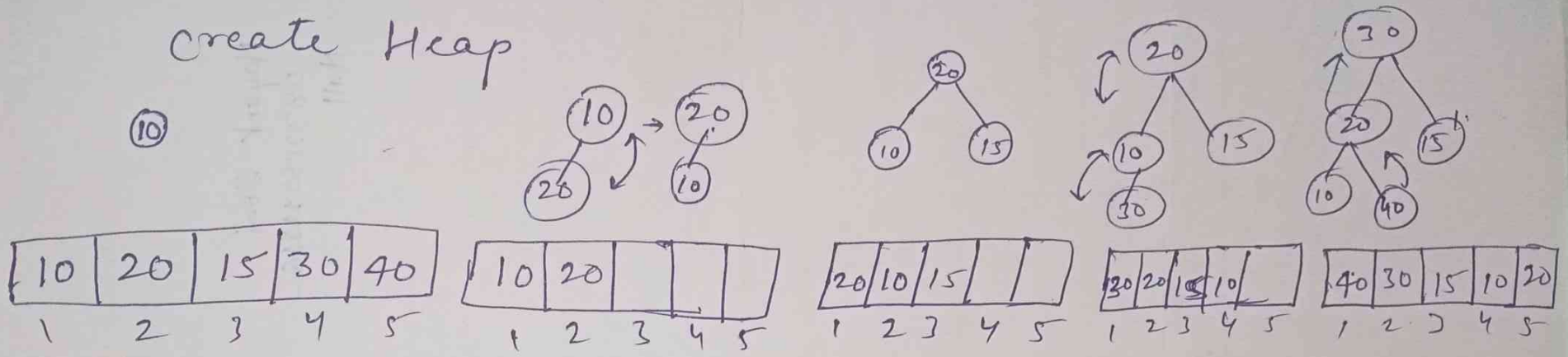
Max Heap



Max Heap

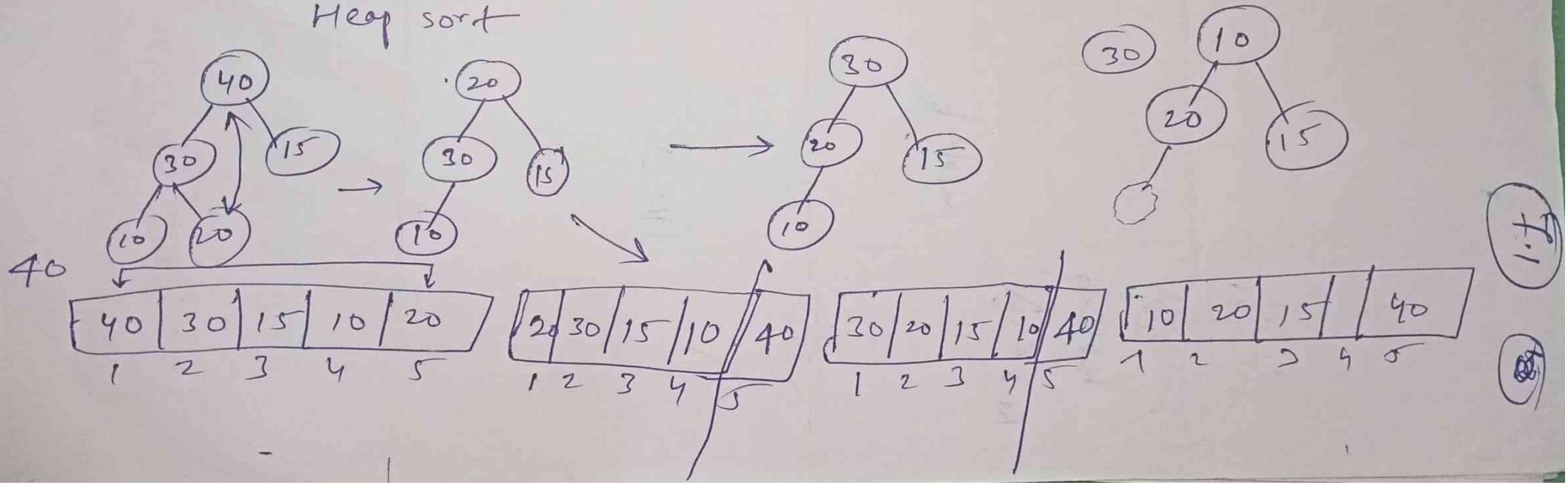
Heap

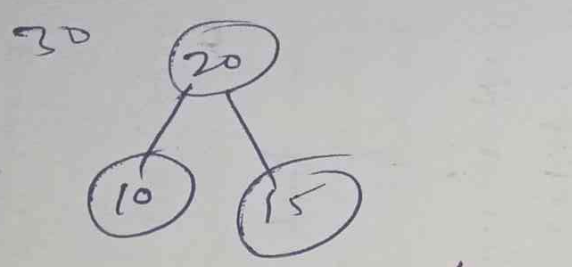
Create Heap

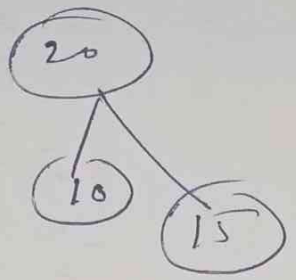


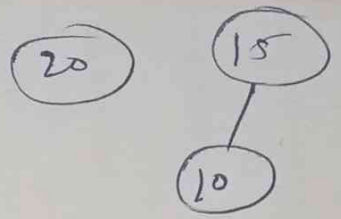
$n \log n$

Heap sort

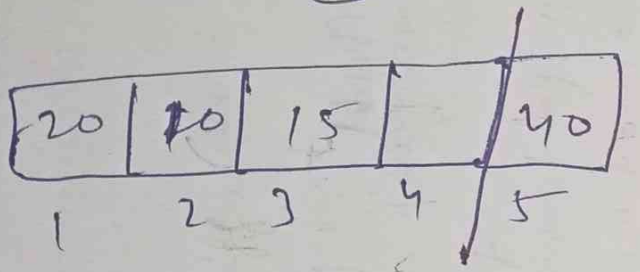


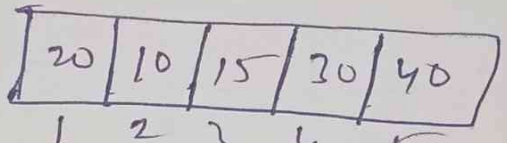

30
20
10
15

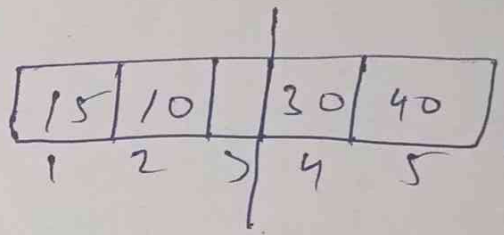

20
10
15

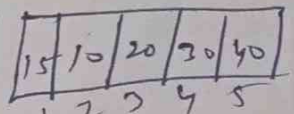

20
15
10

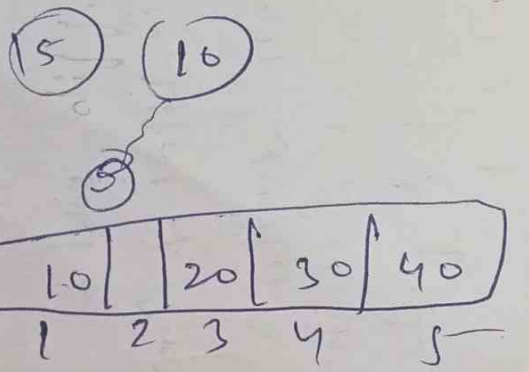

15
10

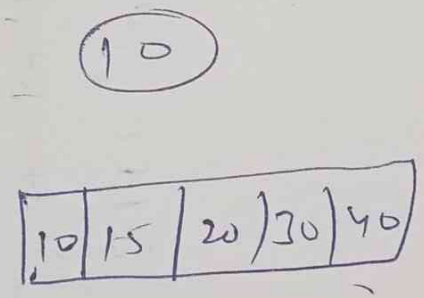

20
10
15
40
1
2
3
4
5


20
10
15
30
40
1
2
3
4
5


15
10
30
40
1
2
3
4
5


15
10
20
30
40
1
2
3
4
5


5
10
5
10
20
30
40
1
2
3
4
5

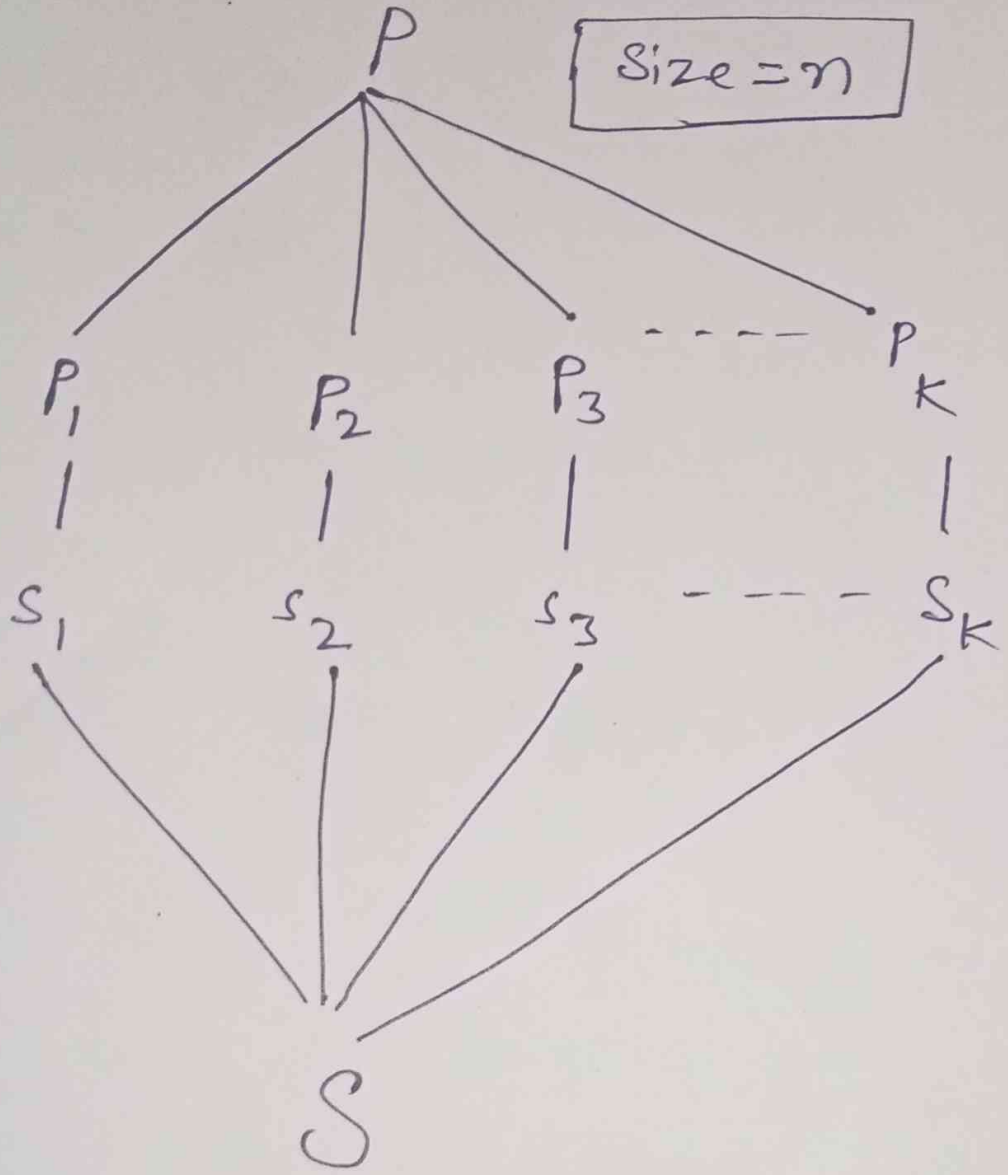

10
10
15
20
30
40

7.2

Algorithm strategies

1. Divide & Conquer
2. Greedy Method
3. Dynamic programming
4. Backtracking

Divide and Conquer



Divide and Conquer

9.1

```
DAC (P)
{
if (small(P))
   {
     S(P);
   }

else
   {
     divide P into P1, P2, P3 --- Pk
     Apply DAC (P1), DAC(P2), ---
     Combine (DAC (P1), DAC(P2), --)
   }

}
```

9.2

Divide & Conquer Applied Here

1. Binary search
2. Merge sort
3. Quick sort
4. Strassen's matrix multiplication

Recurrence Relation

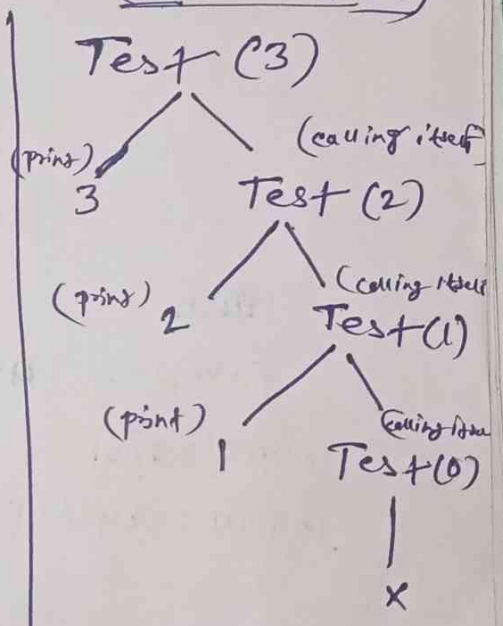
```
T(n) - void Test (int n)
       {
         if (n > 0)
           {
1 ------    printf ("%d", n);
T(n-1) ---  Test (n-1)
           }
       }
```

Test (3)

$T(n) = T(n-1) + 1$

$T(n) = T(n-1) + 1$

(Recursive Tree)



3+1

$f(n) = n+1$ (calls)
n (printf)

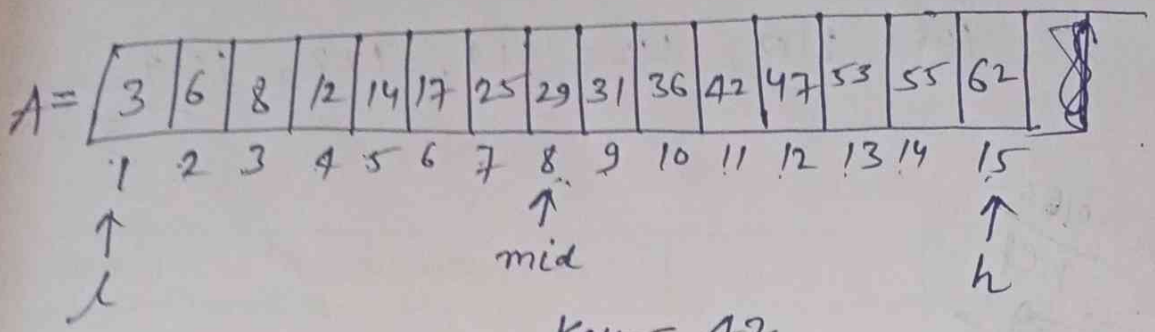
$O(n)$

~~ADA~~ ~~Lab~~ Binary search Algorithm

1. Binary search Algorithm

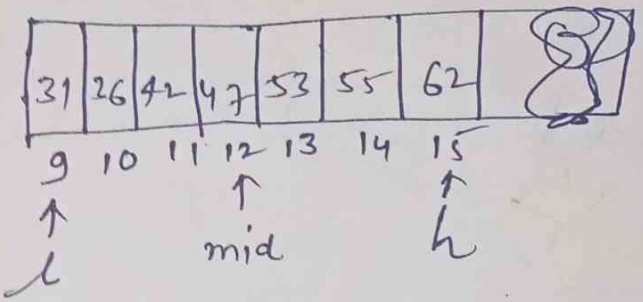
* It works on diride & conquer strategy
* In diride & conquer a problem divides into subproblem & so on.
* ~~To perfo~~ list must be sorted

10.1

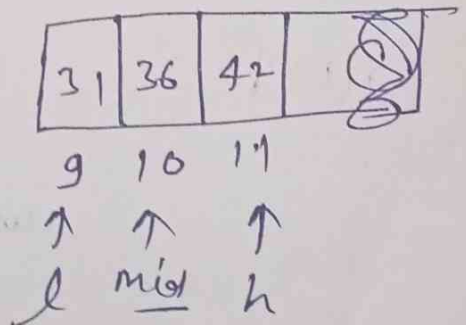


Key = 42

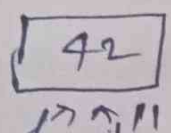
	l	h	$mid = \left\lfloor \frac{l+h}{2} \right\rfloor$
(1) →	1	15	$\frac{1+15}{2} = 8$



(2) →	9	15	$\frac{9+15}{2} = 12$



(3) →	9	11	$\frac{9+11}{2} = 10$



(4) →	11	11	$\frac{11+11}{2} = 11$

Algorithm (Binary search)

```
BinSearch (A, n, key)
{
  l = 1;
  h = n;
while(l ≤ h)
{
  mid = (l+h)/2;
   if (key == A [mid])
      return mid;
   if(key < A [mid])
        h = mid - 1;
      else
        l = mid + 1;
}
return 0;
}
```

Binary search

10.3

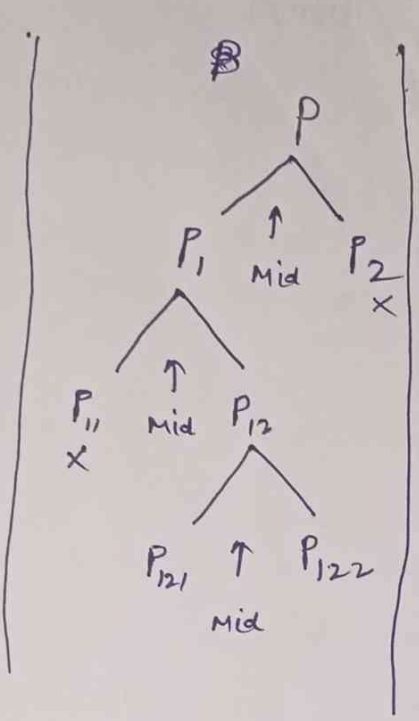
$n = 15$

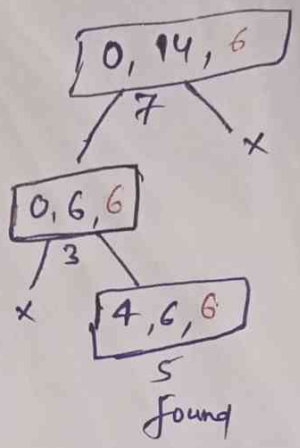
	l													h
A =	-12	-8	-6	-1	3	6	8	11	14	17	20	22	25	29	31
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Key = 6

l	h	mid = (l+h)/2
0	14	(0+14)/2 = 7
0	6	(0+6)/2 = 3
4	6	(4+6)/2 = 5

found
stop





[l==h, l<h, l>h]

```
BinSrch (int l, int h, int key)
{
  if (l > h)
    return -1; // Element not found
  int mid = (l+h)/2;
  if (key == a[mid])
    return mid; // Element found
  else if (key < a[mid])
    return BinSrch (l, mid-1, key);
  else
    return BinSrch (mid+1, h, key);
}
```

10.4

The time Complexity of a binary Search is $O(\log_2 n)$.

This means that as the number of elements (n) in a sorted array grows, the time it takes to find a target value increases very slowly.

Why is it logarithmic?

Binary search uses a divide-and-conquer strategy. Each step eliminates half of the remaining search space.

1. Iteration 1: n elements remaining
2. Iteration 2: $n/2$ elements remaining
3. Iteration 3: $n/4$ (or $n/2^2$) elements remaining
4. Iteration k: $n/2^k$ elements remaining

The search ends when the search space is reduced to 1 element ($n/2^k = 1$)

$n = 2^k \Rightarrow \log_2 n$ $\quad n = \log_2 n$

Merge sort

11

* Merge sort is efficient, comparison based sorting algorithm.
* Based on divide & conquer.

Algorithm

A

3	2	4	1
1	2	3	4

$n \Rightarrow n/2^0$

$n/2 \Rightarrow n/2^1$

Divide $n/4 \Rightarrow n/2^2$

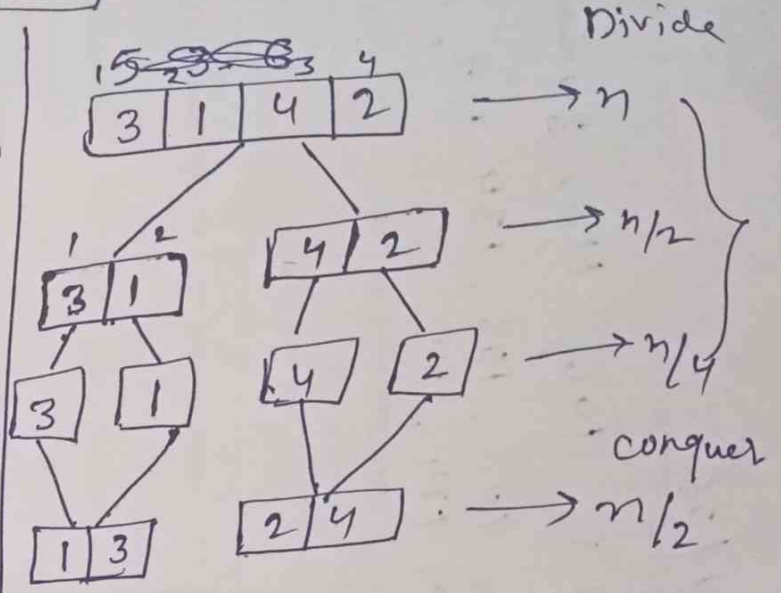
$n/8 \Rightarrow n/2^3 \Rightarrow \log_2 8 = 3$

Conquer n

(13.5)

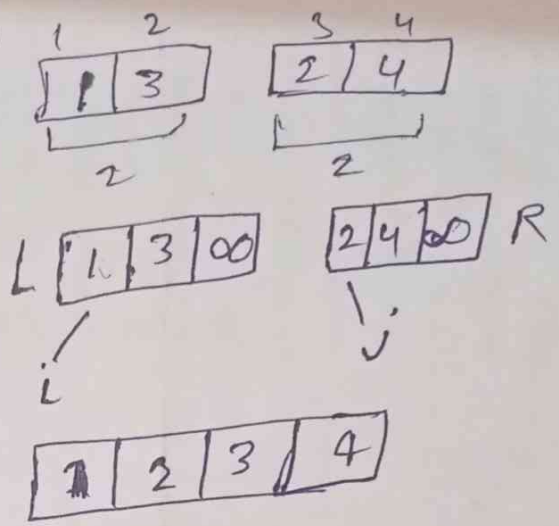
Algorithm Merge_sort

```
Void Merge_Sort (int arr[], int low, int high)
1) if (low < high)
   {
2)  mid = (low+high)/2;
3)  Merge_Sort (arr, low, high);
4)  Merge_Sort (arr, mid+1, high);
5)  Merge (arr, low, mid, high);
```



```
Merge (arr, low, mid, high)
n1 = mid - low + 1;
n2 = high - mid;

Left[n1], Right[n2];
for (i = 0; i < n1; i++)
    Left[i] = arr[low + i];

for (j = 0; j < n2; j++)
    Right[j] = arr[mid + 1 + j];

Left[n1+1] = 9999;
Right[n2+1] = 9999;

i = 0;
j = 0;
k = low;
while (i < n1 && j < n2)
  {
  if (Left[i] <= Right[j])
    {
    arr[k] = Left[i];
      i++;
    }
  else
    {
      arr[k] = Right[j];
        j++;
    }
    k++;
}
```

```
//Copying from Left[]
while (i<n1)
 {
 arr[k] = Left[i];
   i++;
   k++;
}
//Copying from Right[]
while (j<n2)
  {
  arr[k] = Right[j];
     j++;
     k++;
  }
}
```



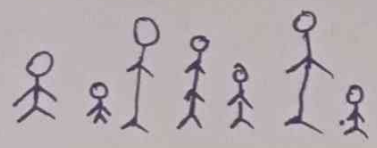
Merge sort Time complexity

$$f(n) = n\log n \times n \Rightarrow n\log n$$

$n\log n$ ↓ Height of Tree

n ↑ Total element to be sort

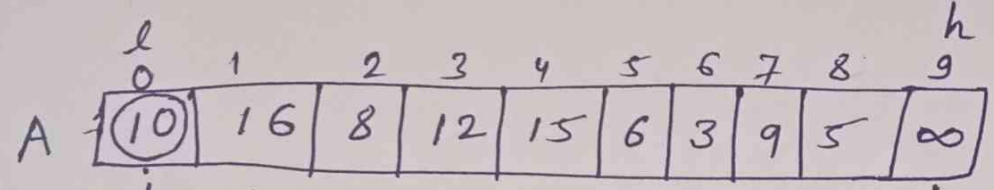
Quick sort Algorithm



(10) 80 90 60 30 20

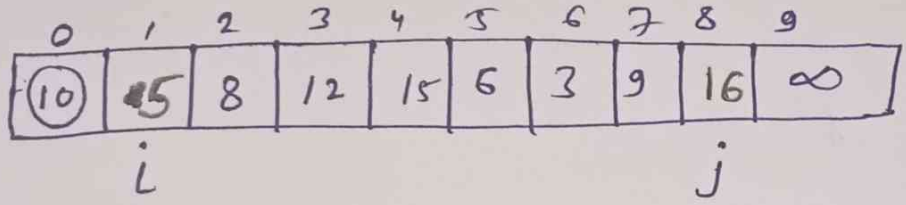
6 3 5 4 2 1 (9)

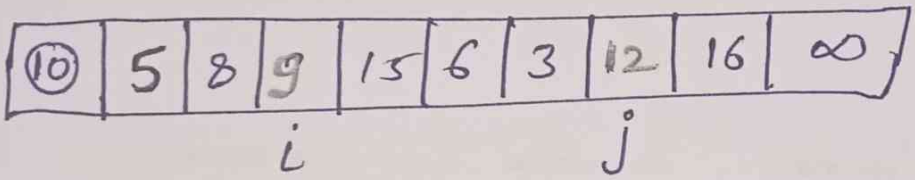
4 6 7 (10) 16 12 13 14

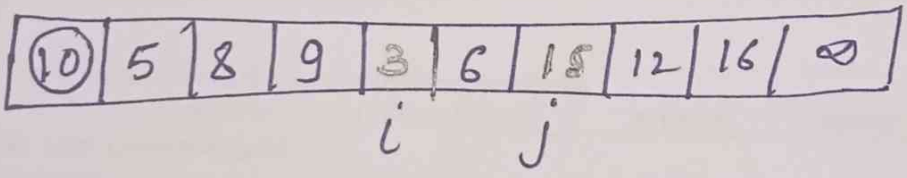


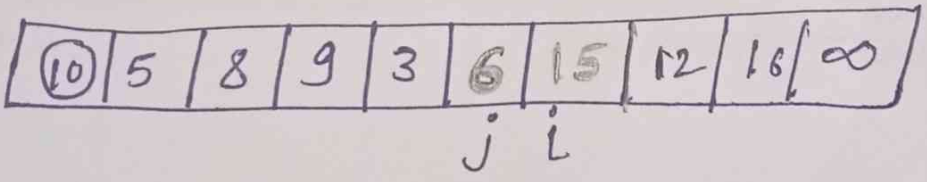
Pivot = 10

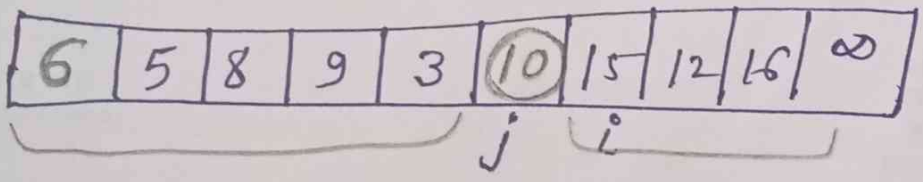
i for >
j for <











~~25~~

12.1

```
Partition (l, h)
{
  pivot = A[l];
  i = l, j = h;
  while (i<j) {
  do
    {
      i++;
    }
  while (A[i] ≤ Pivot);

  do
    {
    j--;
    }
  while (A[j] > Pivot);

  if (i<j)
    {
    swap (A[i], A[j]);
    }
}
 swap (A[l], A[j]);
    return j;
}
```

```
Quick_Sort(l, h)
  {
  if(l<h)
    {
j = Partition(l, h);
Quick_sort(l, j);
Quick_sort (j+1, h);
  }
}
```

Another Example (Quick sort)

Pivot

(50) 70 60 90 40 80 10 20 30 ∞
i j

(50) 70 60 90 40 80 10 20 30 ∞
i j

(50) 30 60 90 40 80 10 20 70 ∞
i i j

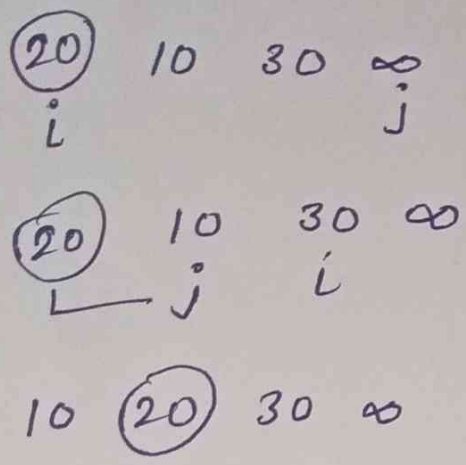
(50) 30 20 90 40 80 10 60 70 ∞
i j

(50) 30 20 10 40 80 90 60 70 ∞
j i

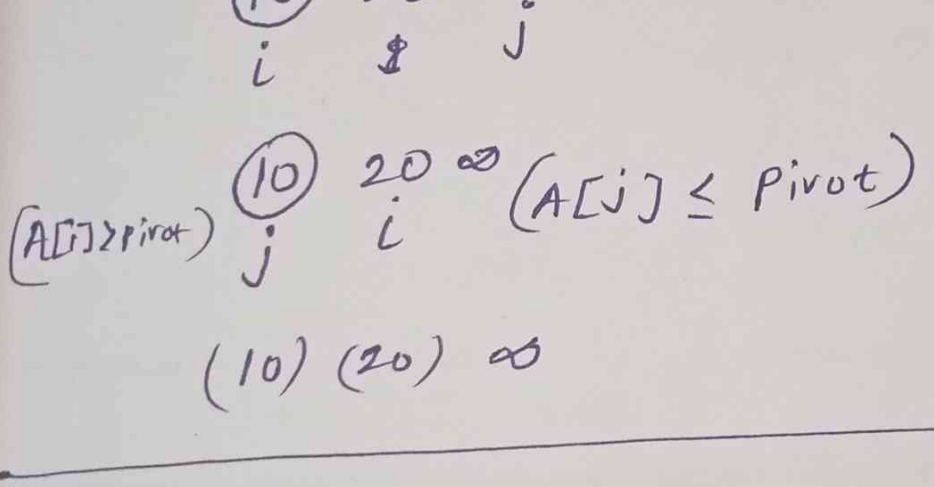
(40 30 20 10) (50) (80 90 60 70 ∞)
j

Partitioning Position

Another Example (Quick sort) (03 elements) 12.3



Another Example (Quick sort)
02 elements



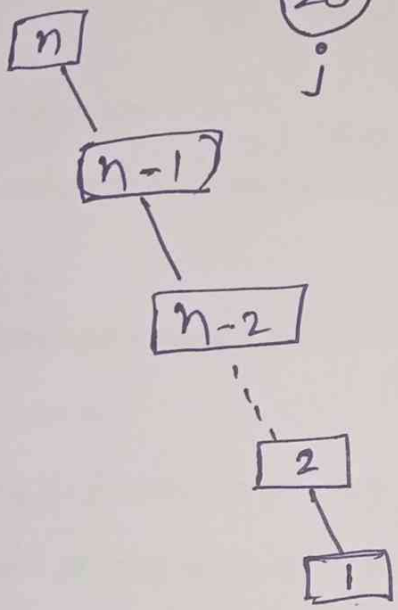
---

worst case

Another Exame (Quick sort) (2)

List is already sorted (Ascending order) (worst case)

(10)	20	30	40	50	∞	
i					j	
(10)	20	30	40	50	∞	n
j	i					
	(20)	30	40	50	∞	n-1
	j	i				
		(30)	40	50	∞	n-2
		j	i			⋮
			(40)	50	∞	2
			j	i		
				50	∞	1



$= 1+2+3--+n$

$= \frac{n(n+1)}{2}$

$= \frac{n^2+n}{2}$

$= O(n^2)$ worst case complexity

12.4

Another Example (Quick sort)

worst case

List sorted in descending order

50 40 30 20 10 ∞

i j

50 40 30 20 10 ∞

j i

10 40 30 20 50 ∞

10 40 30 20 ∞

i j

10 40 30 20 ∞

j i

40 30 20 ∞

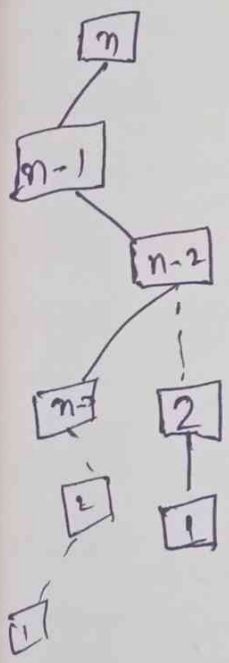
i j

40 30 20 ∞

j i

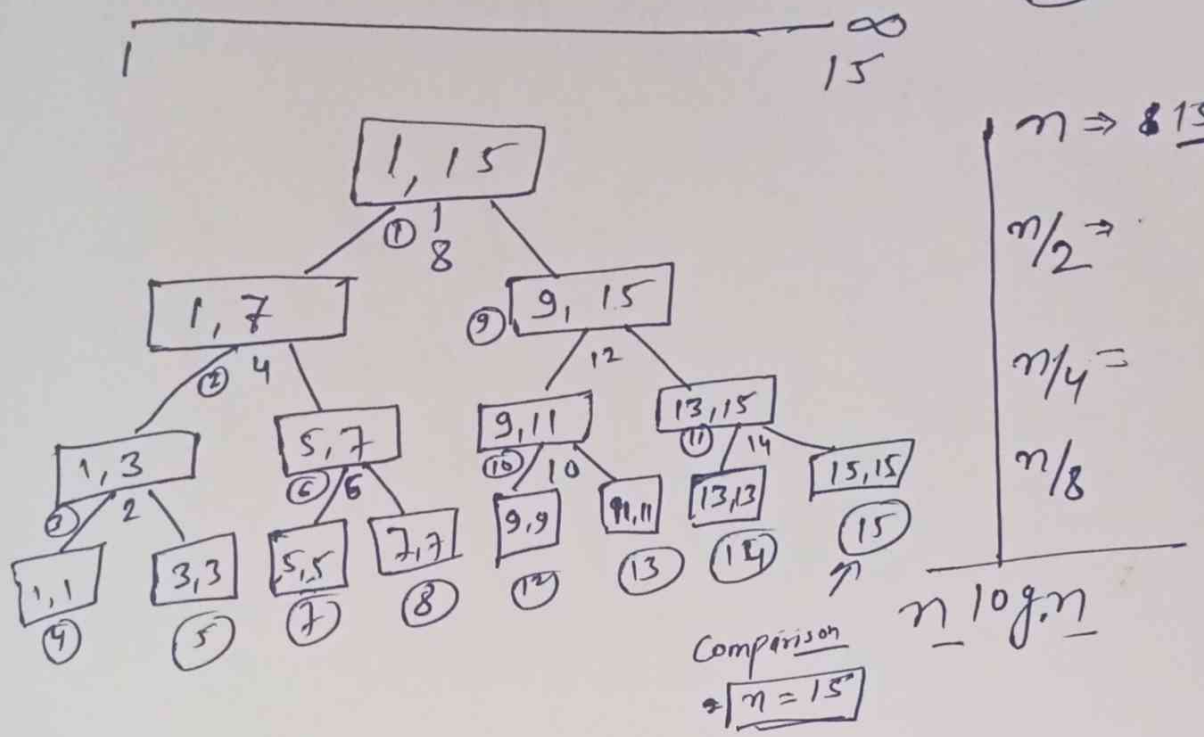
20 30 40 ∞

20



12.5

Example Quick sort (Best case)



① Best case → if partitioning is in middle

Time → $O(n \log n)$

② Worst case → if partitioning is on any end (already sorted)

Time → $O(n^2)$

③ Avg. case → $O(n \log n)$ (you get 1-3 split or 3-1 split)

Another name of Quick sort is Partition exchange sort

12.6

Quick sort (Average case)

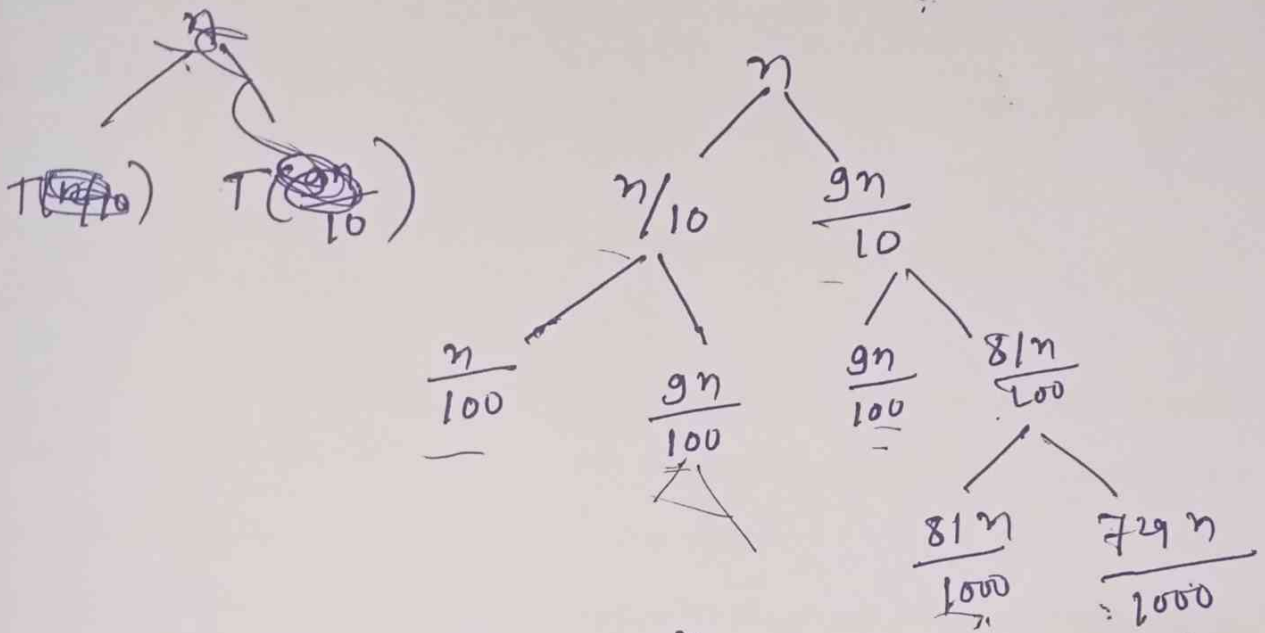
Time complexity

Best case $n/2$ → 02 Time

Worst case $n-1, 0$

Now Average case 9:1

$$T(n) = T(n/10) + T\left(\frac{9n}{10}\right) + n$$



$$n, \frac{n}{(10/9)}, \frac{n}{(10/9)^2}, \frac{n}{(10/9)^3}, \ldots\ldots 1$$

$$\log_{10/9} n \Rightarrow 2^k$$

8^2

2^3

2.81

$$T(n) = n \log_{10/9} n$$

12.7

Normal Matrix Multiplication (DAA)

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

C = m.p

2×2 $\quad$ 2×2 $\quad$ 2×2

$m \times n$ $\quad$ $m \times p$ $\quad$ $m \times p$

No. of column in the first Matrix = No. of row in second Matrix

$$C_{ij} = \sum_{k=1}^{n} A_{ik} * B_{kj}$$

```
n+1 ---- for (i=0; i<=n; i++)
         {
(n+1)xn ---- for (j=0; j<=n; j++)
             {
nxn ---------- C[i,j] = 0;
(n+1)xnxn ---- for (k=0; k<=n; k++)
               {
nxnxn -------- C[i,j] += A[i,k] * B[k,j];
               }
             }
         }
```

$f(n) = 2n^3 + 3n^2 + 2n + 1$

$O(n^3)$

Time Complexity $T(n) = O(n^3)$

Space Complexity

A - n^2

B - n^2

C - n^2

n - 1

i = 1

j = 1

k = 1

$S(n) = 3n^2 + 4$

$O(n^2)$

Normal

13.1

Matrix Multiplication (DAC)

$$A = \underset{2\times2}{\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}} \times B = \underset{2\times2}{\begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}} \Rightarrow C = \underset{2\times2}{\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}}$$

~~$c_{11} = a_{11} * b_{11} + a_{21} * b_{11}$~~

$$c_{11} = a_{11} * b_{11} + a_{12} * b_{21}$$

$$c_{12} = a_{11} * b_{12} + a_{12} * b_{22}$$

$$c_{21} = a_{21} * b_{11} + a_{22} * b_{21}$$

$$c_{22} = a_{21} * b_{12} + a_{22} * b_{22}$$

$$A = \underset{1\times1}{[a_{11}]} \quad B = \underset{1\times1}{[b_{11}]}$$

$$C = \underset{1\times1}{[a_{11} * b_{11}]}$$

13.1

Divide & Conquer

13.2

Matrix Multiplication (DAC)

$$A = \left[\begin{array}{cc|cc} a_{11} & a_{12} & a_{13} & a_{14} \\ & A_{11} & & A_{12} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ \hline a_{31} & a_{32} & a_{33} & a_{34} \\ & A_{21} & & A_{22} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{array}\right] \qquad B = \left[\begin{array}{cc|cc} b_{11} & b_{12} & b_{13} & b_{14} \\ & B_{11} & & B_{12} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ \hline b_{31} & b_{32} & b_{33} & b_{34} \\ & B_{21} & & B_{22} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{array}\right]$$

$\frac{4}{2} \times \frac{4}{2}$ $\qquad$ $\frac{4}{2} \times \frac{4}{2}$

$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$

$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$

$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$

$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$

```
Algorithm MM(A, B, n)
{
  if (n <= 2)
  {
    C = 4 formulas;
    .
    .
    .
  }
  else
  {
    mid -- n/2
    MM(A11, B11, n/2) + MM(A12, B21, n/2)
    MM(A11, B12, n/2) + MM(A12, B22, n/2)
    MM(A21, B11, n/2) + MM(A22, B21, n/2)
    MM(A21, B12, n/2) + MM(A22, B22, n/2)
  }
}
```

13.2

DAC

Matrix Multiplication

(Recurrence Relation)

$$T(n) = \begin{cases} b & n \leq 2 \\ 8\,T(n/2) + n^2 & n > 2 \end{cases}$$

$8^2 = 16$

As per Master's Theorem:

$a = 8$

$b = 2$

$f(n) = n^2$

$$\boxed{n^{\log_b^a} = n^{\log_2^8} = n^3}$$

$n^k = n^2$

$\Theta\left(n^{\log_b a}\right) \Rightarrow \Theta(n^3)$ Normal Matrix Multiplication

---

Master's Theorem

$$T(n) = aT(n/b) + f(n) \quad a \geq 1, \; b > 1$$

It compares the cost of work done at the top level $f(n)$ with the cost of work done at the leaves $(n^{\log_b a})$

13.4

152

What Strassen has done?

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22}) B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$$

$$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$$

$$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$$

$$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

$$T(n) = \begin{cases} b & n \le 2 \\ 7\,T(n/2) + n^2 & n > 2 \end{cases}$$

~~for = O(n log~~

$n^{\log_2 7} = n^{2.81}$

for $k = 2$

$O(n^{\log_2 7}) = O(n^{2.81})$

(By using master's ~~theo~~ theorem $a = 7$ $b = 2$

13.5

$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$ Strassen's Method

$P = (A_{11} + A_{22})(B_{11} + B_{22})$ ← Diagonal Multiply

$Q = B_{11}(A_{21} + A_{22})$

$R = A_{11}(B_{12} - B_{22})$

$S = A_{22}(B_{21} - B_{11})$

$T = B_{22}(A_{11} + A_{12})$

$U = (A_{21} - A_{11})(B_{11} + B_{12})$

$V = (B_{21} + B_{22})(A_{12} - A_{22})$

T(+), S(-), R(-), Q(+)

$B_{11}\ A_{11}\ A_{22}\ B_{22}$

Replace
A → B
B → A
Replace

Now calculate C_{ij} — V @ first

Add

③ $C_{11} = P + S - T + V$

① $C_{12} = R + T$

② $C_{21} = Q + S$

④ $C_{22} = P + R - Q + U$

RAT formula

$(R-1) + (T-1)$

U at last

Just write P @ start

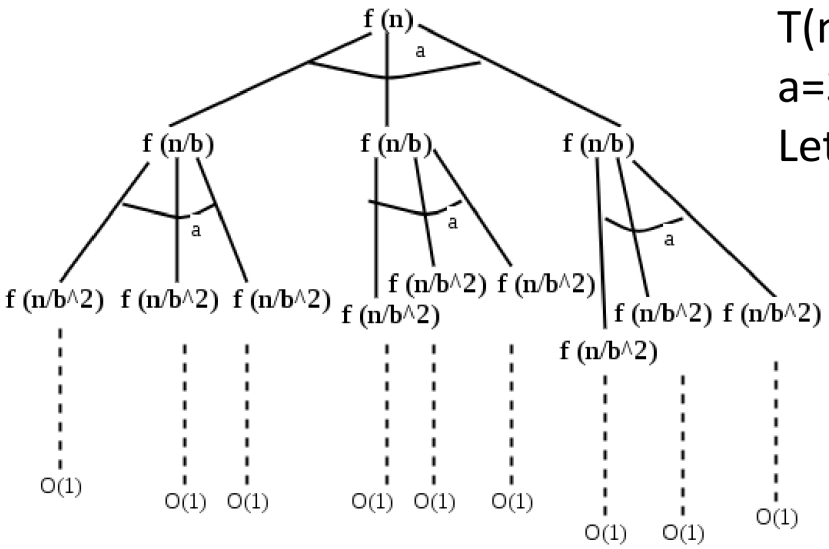
How does the Master's theorem work?

The master method is mainly derived from the recurrence tree method. If we draw the recurrence tree of **T(n) = aT(n/b) + f(n),** we can see that the work done at the root is f(n), and work done at all leaves is $O(n^c)$ where c is $Log_b a$ and the height of the recurrence tree is $Log_b n$

Key Points

- The Master Theorem is specifically designed for divide-and-conquer recurrences
- It applies only to recurrences of the form: **T(n) = aT(n/b) + f(n)**
- It provides a quick and reliable way to compute asymptotic time complexity
- Not all recurrences can be solved using this method
- For unsupported cases, we either use Substitution Method or Recurrence Tree Method

Recurrence Tree Method



T(n) = aT(n/b) + f(n)

T(n) = 3T(n/3)+n

a=3, b=3

Let's assume n=27

Substitution Method

Let’s Solve the following recurrence relation using substitution method
T(n)=3T(n/3)+n

1. Expand the recurrence relation

Begin by repeatedly substituting the recurrence into itself to observe the developing pattern.

- **First expansion:**

$$T(n) = 3T\left(\frac{n}{3}\right) + n$$

- **Second expansion:**
  Substitute $T(n/3) = 3T(n/9) + n/3$:

$$T(n) = 3\left[3T\left(\frac{n}{9}\right) + \frac{n}{3}\right] + n = 9T\left(\frac{n}{9}\right) + n + n = 9T\left(\frac{n}{9}\right) + 2n$$

- **Third expansion:**
  Substitute $T(n/9) = 3T(n/27) + n/9$:

$$T(n) = 9\left[3T\left(\frac{n}{27}\right) + \frac{n}{9}\right] + 2n = 27T\left(\frac{n}{27}\right) + n + 2n = 27T\left(\frac{n}{27}\right) + 3n$$

2. Identify the general pattern

After k substitutions, the relation takes the general form:

$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + k \cdot n$$

3. Apply base case boundary

The iteration stops when the subproblem size reduces to the base case, typically $T(1)$.
Set the argument of T to 1:

$$\frac{n}{3^k} = 1 \Longrightarrow n = 3^k \Longrightarrow k = \log_3 n$$

4. Substitute k back into the formula

Replace k with $\log_3 n$ and 3^k with n in the general form:

$$T(n) = n \cdot T(1) + (\log_3 n) \cdot n$$

$$T(n) = n \cdot T(1) + n \log_3 n$$

Since $T(1)$ is a constant, the term $n\log_3 n$ dominates the growth.

✅ Solution

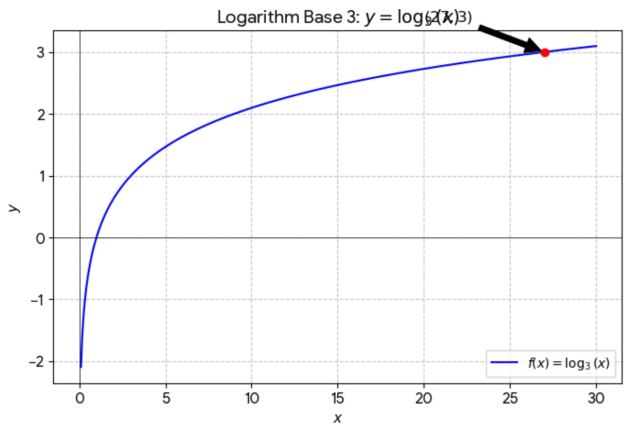
The complexity of the recurrence relation is $\boldsymbol{T(n) = \Theta(n \log n)}$.

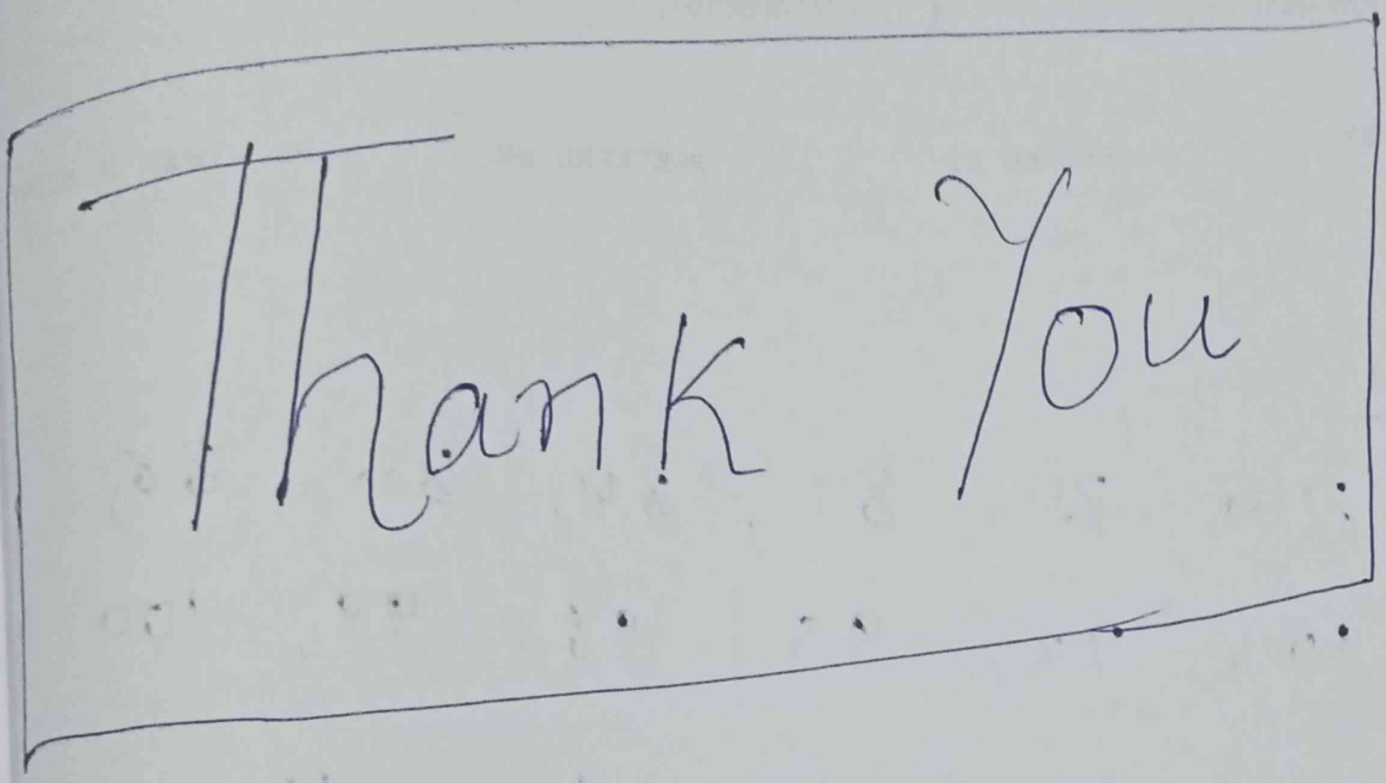
Applying the math from the image:

If we plug $n = 27$ into the equations from your image, they all match up perfectly:

1. **The Fraction:** $\frac{27}{3^k} = 1$
2. **The Power:** $27 = 3^k$ (This asks: "3 to what power is 27?")
3. **The Logarithm:** $k = \log_3 27$

Since $3 \times 3 \times 3 = 27$, we know that $3^3 = 27$. Therefore, $\boldsymbol{k = 3}$.




Thank You